Componentes de interface

Flutter

Prof. Cristiano Rodrigues

Hello, world

Sem usar widget

Hello, world!

- Crie um aplicativo conforme código ao lado.
- Substitua todo o texto do arquivo main.dart pelo código ao lado.

```
1   import 'package:flutter/material.dart';
2
    Run | Debug | Profile
3   void main() {
4     runApp(
5       const MaterialApp(
6         title: 'Exemplo',
7         home: Scaffold(
8           body: Center(
9             child: Text("Hello World")) // Center
10        ) // Scaffold
11      ) // MaterialApp
12    );
13  }
```

Hello, world!

- O aplicativo anterior funciona mas não é usual criarmos daquela forma.

- O padrão é criarmos widgets customizados a partir do StatelessWidget ou do StatefulWidget.

Hello, world!

- Crie um arquivo main.dart contendo o código ao lado.
- Em seguida, crie um novo arquivo dentro da pasta lib chamado hello_page.dart contendo o código do slide a seguir.
- Teste o aplivativo.

```
import 'package:flutter/material.dart';
import 'hello_page.dart';

void main() {
        runApp(const Hello());
}
```

```dart
import 'package:flutter/material.dart';

class Hello extends StatelessWidget {
	const Hello({super.key});

	@override
	Widget build(BuildContext context) {
		return MaterialApp(
			title: 'Atividade Pratica’,
			theme: ThemeData(
				colorScheme: ColorScheme.fromSeed(
					seedColor: const Color.fromARGB(
						255,58, 183, 93)),
				useMaterial3: true,
			),
			home: const Scaffold(
				body: Center(child: Text("Hello, world"))));
	}
}
```

Hello, world!

- Teste o aplicativo.

- Essa é a forma usual de criarmos um aplicativo, sempre construindo Widgets customizados a partir do StatelessWidget ou do StatefulWidget.

Componentes de interface

O que vamos ver

- MaterialApp
- Scaffold
- StatelessWidget
- Text/TextField
- ElevatedButton
- Imagens

Material Design

- Agora vamos olhar os componentes usados na nossa aplicação

```
import 'package:flutter/material.dart';

void main() {
  runApp(
      const MaterialApp(
        title: 'Flutter Demo',
        home: Scaffold(
          body: Center(
            child: Text("Hello World"))  // Center
        )  // Scaffold
      )  // MaterialApp
  );
}
```

Material Design[1]: É é uma "linguagem de design" orientada para Android e criada pelo Google

[1] Disponível em https://material.io/

MaterialApp()

- Agora vamos olhar os componentes usados na nossa aplicação

```
import 'package:flutter/material.dart';

void main() {
  runApp(
      const MaterialApp(
        title: 'Flutter Demo',
        home: Scaffold(
          body: Center(
            child: Text("Hello World"))  // Center
        )  // Scaffold
      )  // MaterialApp
  );
}
```

MaterialApp: Cria a estrutura externa para seu aplicativo. Ele envolve toda a sua aplicação, e permite atribuir um **título** a ela assim como um **tema padrão**, com fontes, tamanhos, cores e assim por diante.

MaterialApp()

- Agora vamos olhar os componentes usados na nossa aplicação

title: título da aplicação

```
import 'package:flutter/material.dart';

void main() {
  runApp(
      const MaterialApp(
        title: 'Flutter Demo',
        home: Scaffold(
          body: Center(
            child: Text("Hello World"))  // Center
        )  // Scaffold
      )  // MaterialApp
  );
}
```

MaterialApp()

- Agora vamos olhar os componentes usados na nossa aplicação

```
import 'package:flutter/material.dart';

void main() {
  runApp(
      const MaterialApp(
        title: 'Flutter Demo',
        home: Scaffold(
          body: Center(
            child: Text("Hello World"))  // Center
        )  // Scaffold
      )  // MaterialApp
  );
}
```

title: título da aplicação

home: a primeira tela do App deve ser especificada dentro desse atributo. Telas flutuantes são distribuídas em rotas. Uma “rota” no Flutter é apenas uma única tela.

MaterialApp()

- Agora vamos olhar os componentes usados na nossa aplicação

```
import 'package:flutter/material.dart';

void main() {
  runApp(
      const MaterialApp(
        title: 'Flutter Demo',
        home: Scaffold(
          body: Center(
            child: Text("Hello World"))  // Center
        )  // Scaffold
      )  // MaterialApp
  );
}
```

title: título da aplicação

home: a primeira tela do App deve ser especificada dentro desse atributo. Telas flutuantes são distribuídas em rotas. Uma "rota" no Flutter é apenas uma única tela.

theme: também é possível escolher um tema para ser usado em toda a aplicação

No exemplo, o App não usa o argumento **theme**

Scaffold()

- Agora vamos olhar os componentes usados na nossa aplicação

```
import 'package:flutter/material.dart';

void main() {
  runApp(
      const MaterialApp(
        title: 'Flutter Demo',
        home: Scaffold(
          body: Center(
            child: Text("Hello World"))  // Center
        )  // Scaffold
      )  // MaterialApp
  );
}
```

Scaffold: cria a estrutura visível interna. Dispõe a estrutura visível da sua aplicação para dar a ela um **layout** previsível e, portanto, utilizável.

Permite criar, entre outras coisas:

- Uma seção para o cabeçalho (**appBar**)
- Uma seção para o corpo (**body**)
- Uma **barra de navegação** na parte inferior ou à esquerda

Scaffold()

- Agora vamos olhar os componentes usados na nossa aplicação

```
import 'package:flutter/material.dart';

void main() {
  runApp(
      const MaterialApp(
        title: 'Flutter Demo',
        home: Scaffold(
          body: Center(
            child: Text("Hello World"))  // Center
        )  // Scaffold
      )  // MaterialApp
  );
}
```

Scaffold: cria a estrutura visível interna. Dispõe a estrutura visível da sua aplicação para dar a ela um layout previsível e, portanto, utilizável.

Permite criar, entre outras coisas:

- Uma seção para o cabeçalho (**appBar**)
- Uma seção para o corpo (**body**)
- Uma **barra de navegação** na parte inferior ou à esquerda

No exemplo, o App não possui **appBar** nem **barra de navegação**

Parâmetros de uma função

```
import 'package:flutter/material.dart';

void main() {
  runApp(
      const MaterialApp(
        title: 'Flutter Demo',
        home: Scaffold(
          body: Center(
            child: Text("Hello World"))  // Center
        )  // Scaffold
      )  // MaterialApp
  );
```

Ctrl + P ou Command + P para ver quais são os parâmetros de uma função. Por exemplo do **MaterialApp()**

{**key**, navigatorKey, scaffoldMessengerKey, home, Map<String, WidgetBuilder> routes: const <String, WidgetBuilder>{}, initialRoute, onGenerateRoute, onGenerateInitialRoutes, onUnknownRoute, List<NavigatorObserver> navigatorObservers: const <NavigatorObserver>[], builder, title: '', onGenerateTitle, color, theme, darkTheme, highContrastTheme, highContrastDarkTheme, themeMode: ThemeMode.system, themeAnimationDuration: kThemeAnimationDuration, themeAnimationCurve: Curves.linear, locale, localizationsDelegates, localeListResolutionCallback, localeResolutionCallback, supportedLocales: const <Locale>[Locale('en', 'US')], debugShowMaterialGrid: false, showPerformanceOverlay: false, checkerboardRasterCacheImages: false, checkerboardOffscreenLayers: false, showSemanticsDebugger: false, debugShowCheckedModeBanner: true, shortcuts, actions, restorationScopeId, scrollBehavior, useInheritedMediaQuery: false}

StatelessWidget

O **StatelessWidget** é um widget **sem estado**, sendo assim, não sofrerá alterações após ser renderizado.

São úteis quando a parte da interface do usuário que está sendo descrita não depende de nada além da informação de configuração no próprio objeto e do contexto **BuildContext** no qual o widget é criado.

```
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Container();
  }
}
```

Criando um Widget customizado

- É possível criar um widget customizado. Para isso, vamos criar uma classe, por exemplo, a classe **HomePage**

```
class HomePage extends StatelessWidget {
  const HomePage({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
```

Criando um Widget customizado

- Todo o widget possui um método **build** que retorna um Widget que é responsável pela tela.

```
class HomePage extends StatelessWidget {
  const HomePage({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
```

Criando um Widget customizado

- Agora recorte o widget **MaterialApp** da função main (do exercício Hello World) e cole na função **build**.

```
class HomePage extends StatelessWidget {
  const HomePage({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      home: Scaffold(
        appBar: AppBar(),
        drawer: const Drawer(),
        body: const Center(...),  // Center
        bottomNavigationBar: BottomNavigationBar(...)
      ),  // Scaffold
    );  // MaterialApp
  }
}
```

Explorando o Scaffold

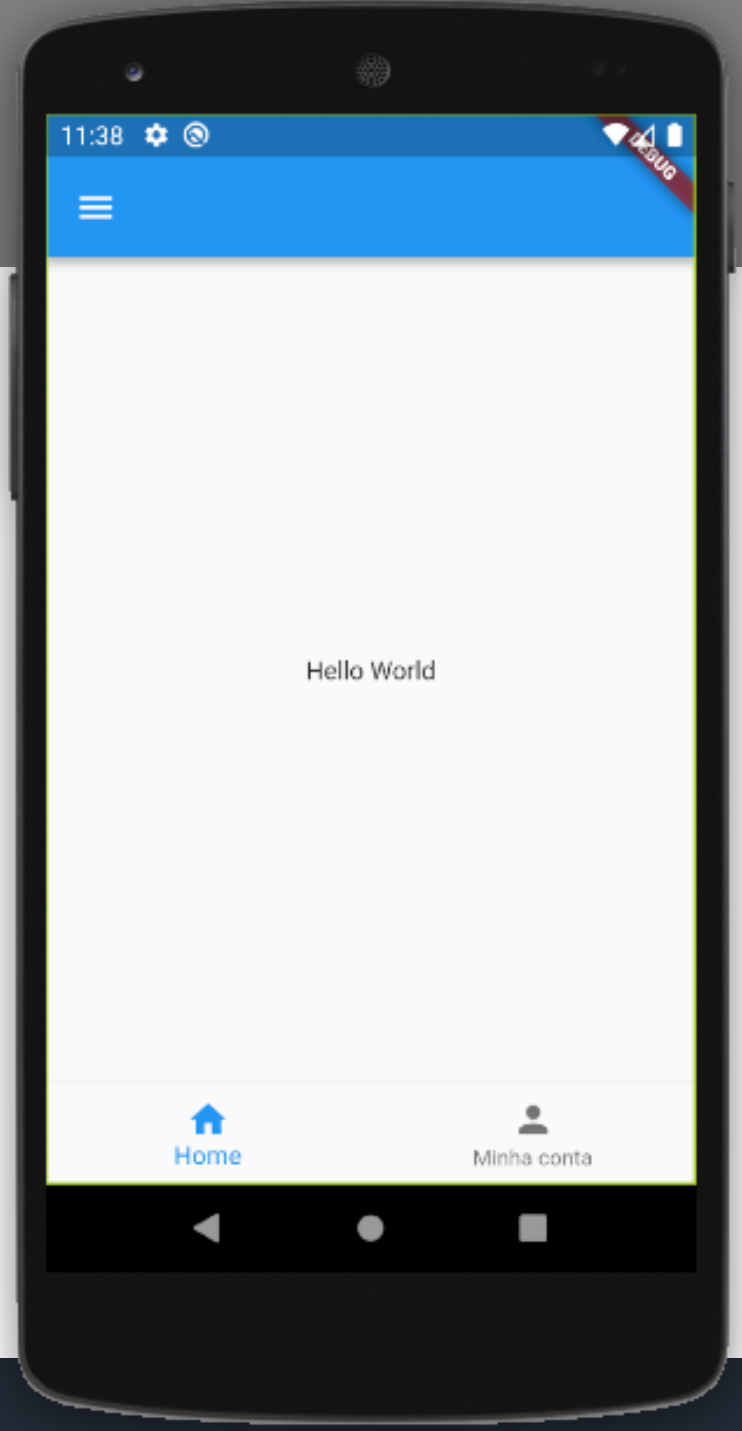
Scaffold()

Vamos explorar algumas opções de **Scaffold**.

Adicione os seguintes argumentos no widget **Scaffold**:

- `appBar: AppBar(),`
- `drawer: Drawer(),`
- `bottomNavigationBar: BottomNavigationBar(`

```
    items: <BottomNavigationBarItem> [
      BottomNavigationBarItem(
        icon: Icon(Icons.home),
        label: 'Home',
      ),  // BottomNavigationBarItem
      BottomNavigationBarItem(
        icon: Icon(Icons.person),
        label: 'Minha conta',
      ),  // BottomNavigationBarItem
    ],),// <BottomNavigationBarItem>[] BottomNavigationBar
```



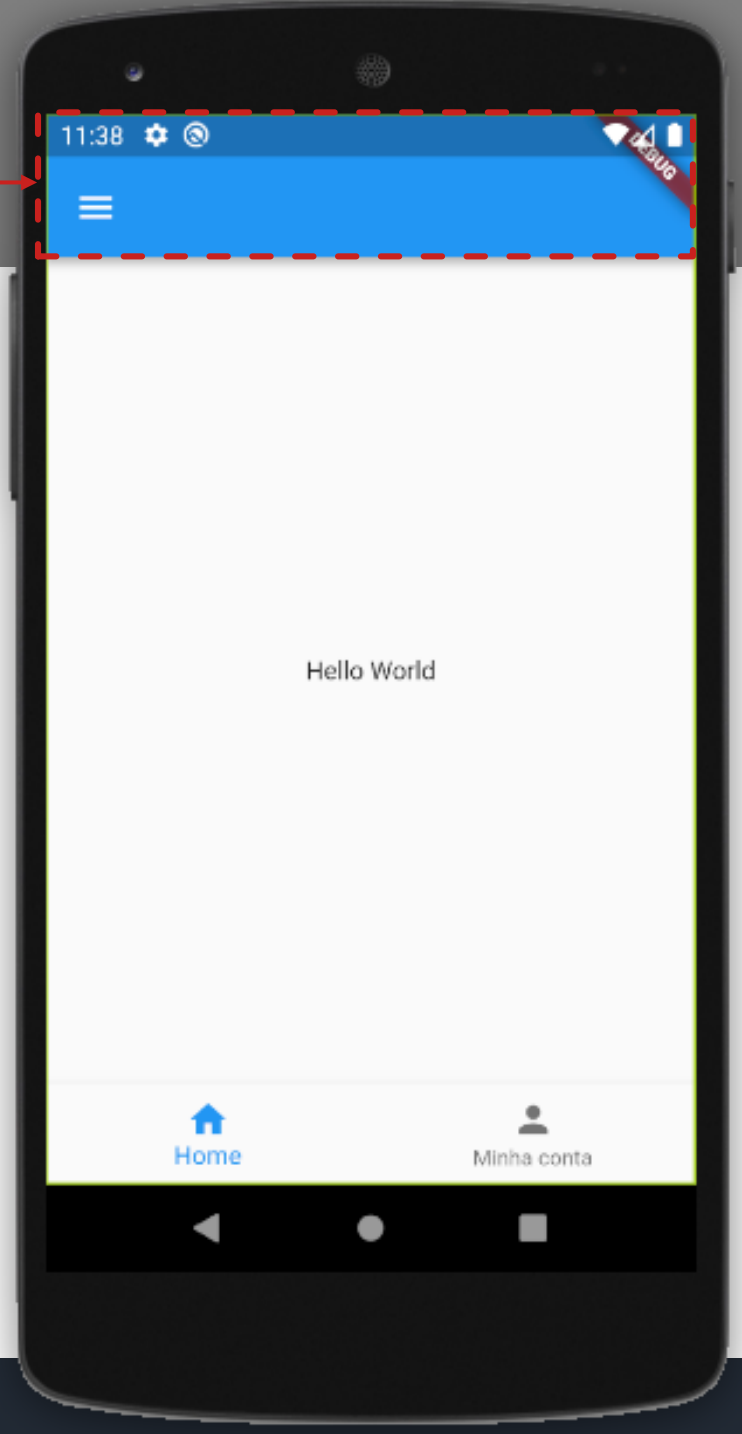
Scaffold()

Vamos explorar algumas opções de **Scaffold**.

Adicione os seguintes argumentos no widget **Scaffold**:

- `appBar: AppBar(),`
- `drawer: Drawer(),`
- `bottomNavigationBar: BottomNavigationBar(`

```
    items: <BottomNavigationBarItem> [
      BottomNavigationBarItem(
        icon: Icon(Icons.home),
        label: 'Home',
      ),  // BottomNavigationBarItem
      BottomNavigationBarItem(
        icon: Icon(Icons.person),
        label: 'Minha conta',
      ),  // BottomNavigationBarItem
    ],),// <BottomNavigationBarItem>[] BottomNavigationBar
```



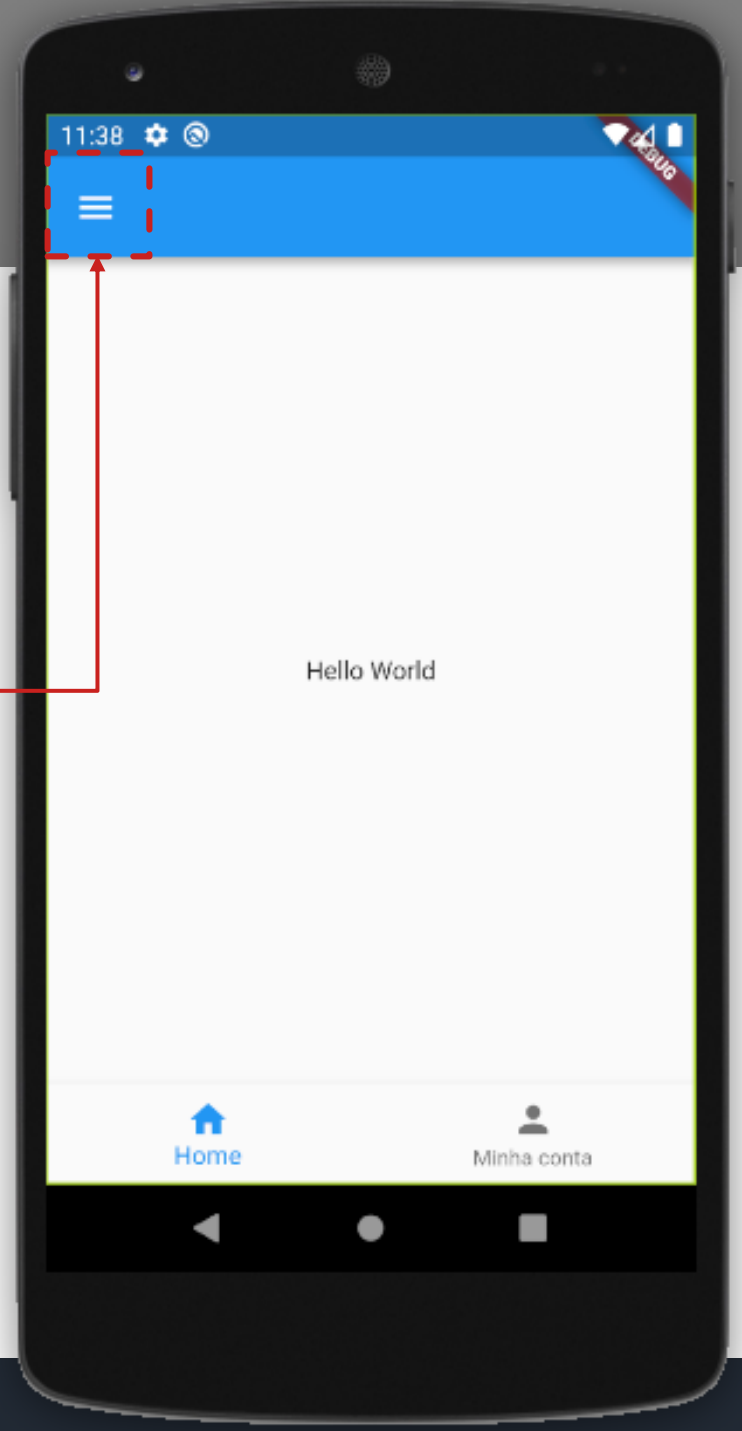
Scaffold()

Vamos explorar algumas opções de **Scaffold**.

Adicione os seguintes argumentos no widget **Scaffold**:

- `appBar: AppBar(),`
- `drawer: Drawer(),`
- `bottomNavigationBar: BottomNavigationBar(`

```
    items: <BottomNavigationBarItem> [
      BottomNavigationBarItem(
        icon: Icon(Icons.home),
        label: 'Home',
      ),  // BottomNavigationBarItem
      BottomNavigationBarItem(
        icon: Icon(Icons.person),
        label: 'Minha conta',
      ),  // BottomNavigationBarItem
    ],),// <BottomNavigationBarItem>[] BottomNavigationBar
```



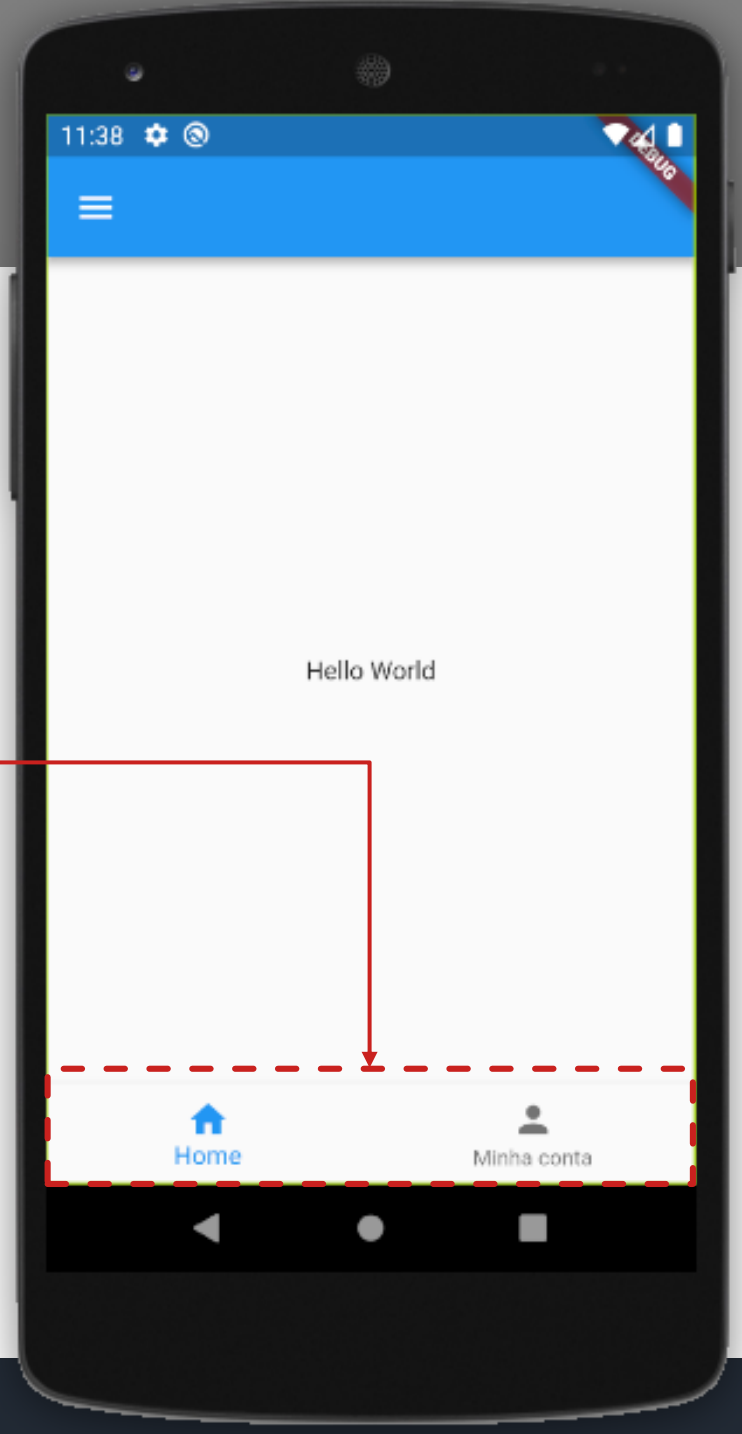
Scaffold()

Vamos explorar algumas opções de **Scaffold**.

Adicione os seguintes argumentos no widget **Scaffold**:

- `appBar: AppBar(),`
- `drawer: Drawer(),`
- `bottomNavigationBar: BottomNavigationBar(`

```
    items: <BottomNavigationBarItem> [
      BottomNavigationBarItem(
        icon: Icon(Icons.home),
        label: 'Home',
      ),  // BottomNavigationBarItem
      BottomNavigationBarItem(
        icon: Icon(Icons.person),
        label: 'Minha conta',
      ),  // BottomNavigationBarItem
    ],),  // <BottomNavigationBarItem>[] BottomNavigationBar
```

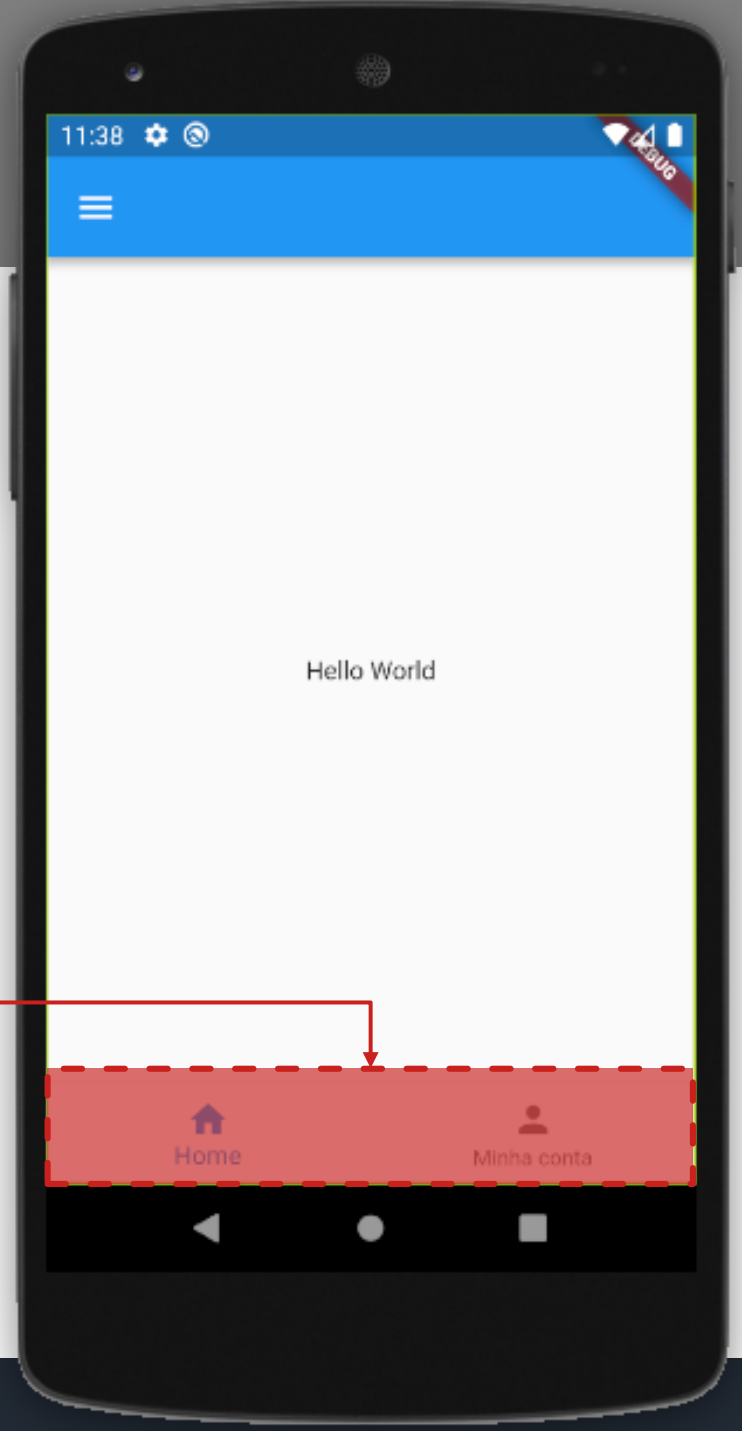


Scaffold()

Vamos explorar algumas opções de **Scaffold**.

Adicione os seguintes argumentos no widget **Scaffold**:

```
appBar: AppBar(),
drawer: Drawer(),
bottomNavigationBar: BottomNavigationBar(
    items: <BottomNavigationBarItem> [
      BottomNavigationBarItem(
        icon: Icon(Icons.home),
        label: 'Home',
      ),  // BottomNavigationBarItem
      BottomNavigationBarItem(
        icon: Icon(Icons.person),
        label: 'Minha conta',
      ),  // BottomNavigationBarItem
    ],),// <BottomNavigationBarItem>[] BottomNavigationBar
```

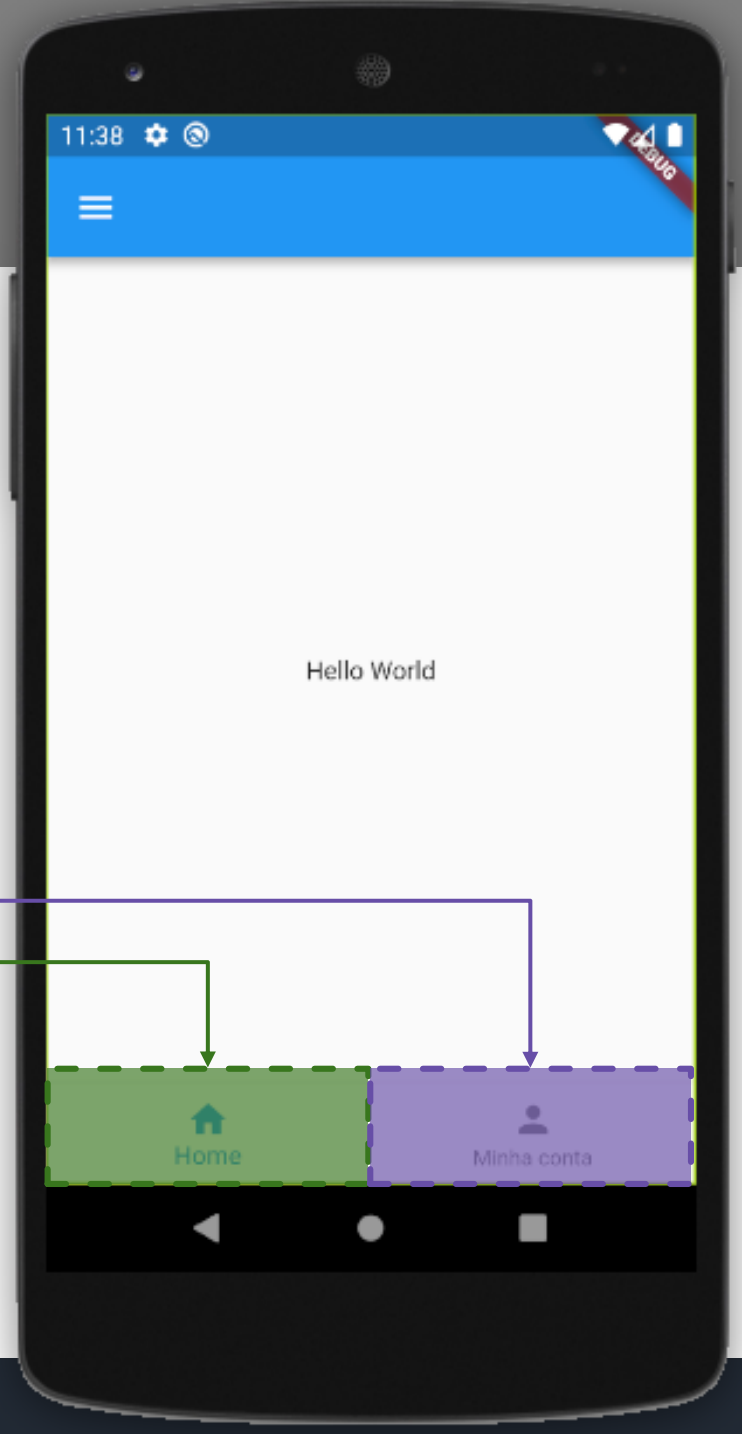


Scaffold()

Vamos explorar algumas opções de **Scaffold**.

Adicione os seguintes argumentos no widget **Scaffold**:

```
appBar: AppBar(),
drawer: Drawer(),
bottomNavigationBar: BottomNavigationBar(
    items: <BottomNavigationBarItem> [
      BottomNavigationBarItem(
        icon: Icon(Icons.home),
        label: 'Home',
      ),  // BottomNavigationBarItem
      BottomNavigationBarItem(
        icon: Icon(Icons.person),
        label: 'Minha conta',
      ),  // BottomNavigationBarItem
    ],),// <BottomNavigationBarItem>[] BottomNavigationBar
```

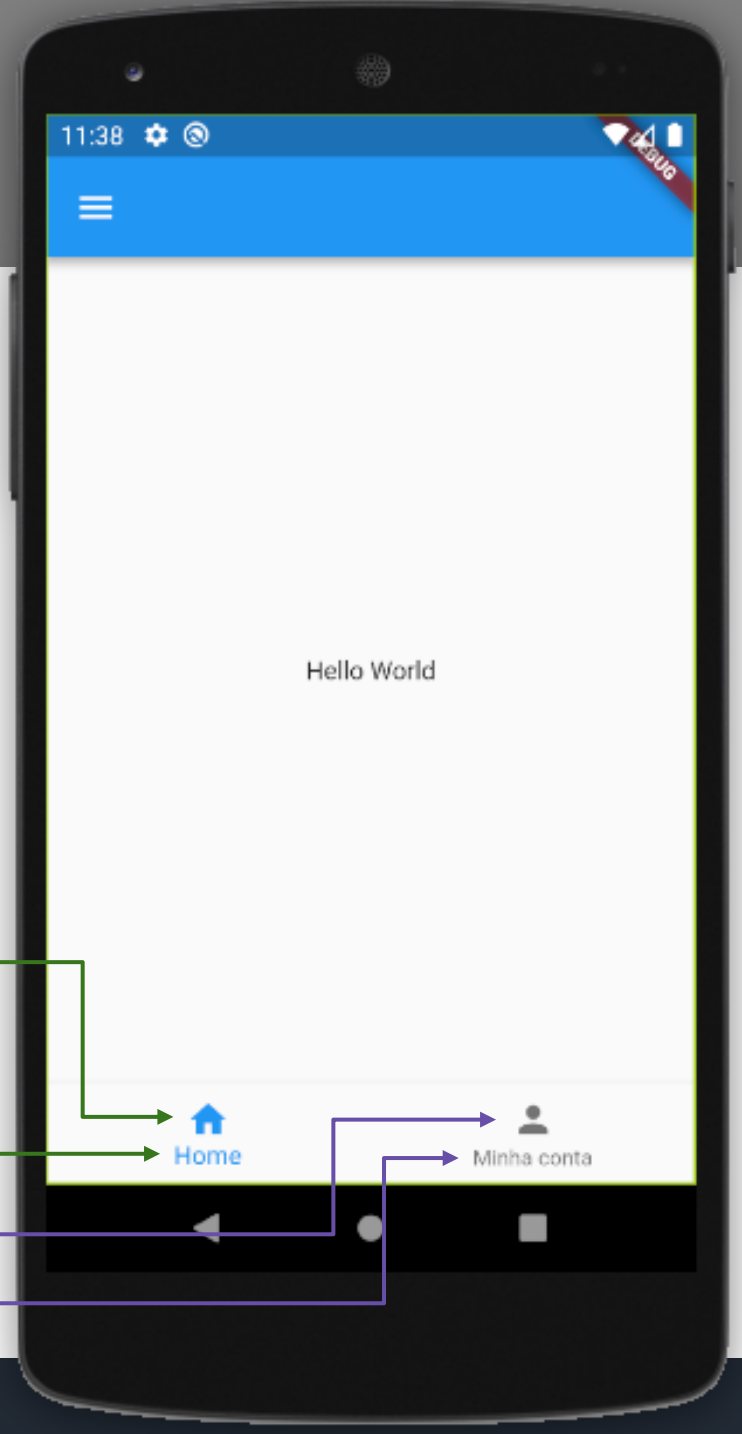


Scaffold()

Vamos explorar algumas opções de **Scaffold**.

Adicione os seguintes argumentos no widget **Scaffold**:

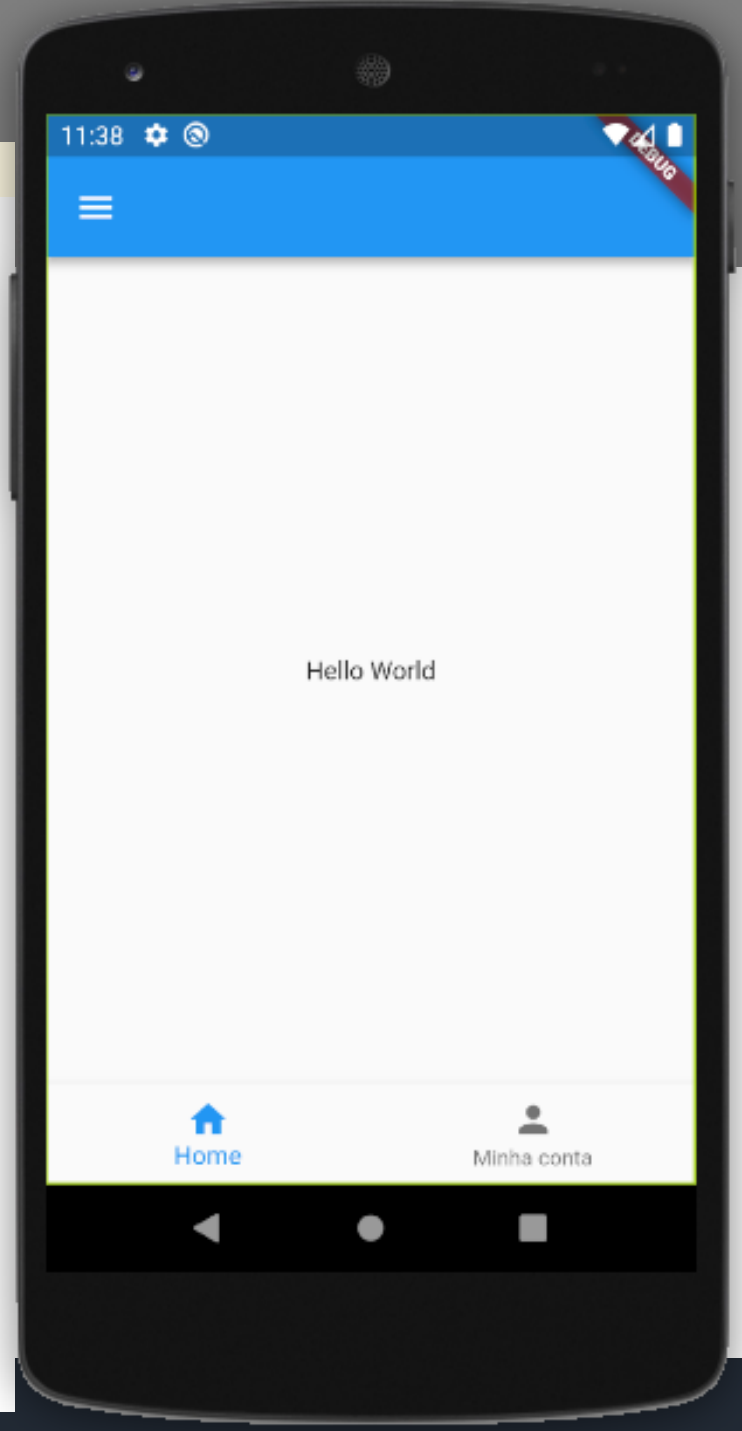
```
appBar: AppBar(),
drawer: Drawer(),
bottomNavigationBar: BottomNavigationBar(
    items: <BottomNavigationBarItem> [
      BottomNavigationBarItem(
        icon: Icon(Icons.home),
        label: 'Home',
      ),  // BottomNavigationBarItem
      BottomNavigationBarItem(
        icon: Icon(Icons.person),
        label: 'Minha conta',
      ),  // BottomNavigationBarItem
    ],),// <BottomNavigationBarItem>[] BottomNavigationBar
```



Scaffold()

Código de exemplo completo do Scaffold

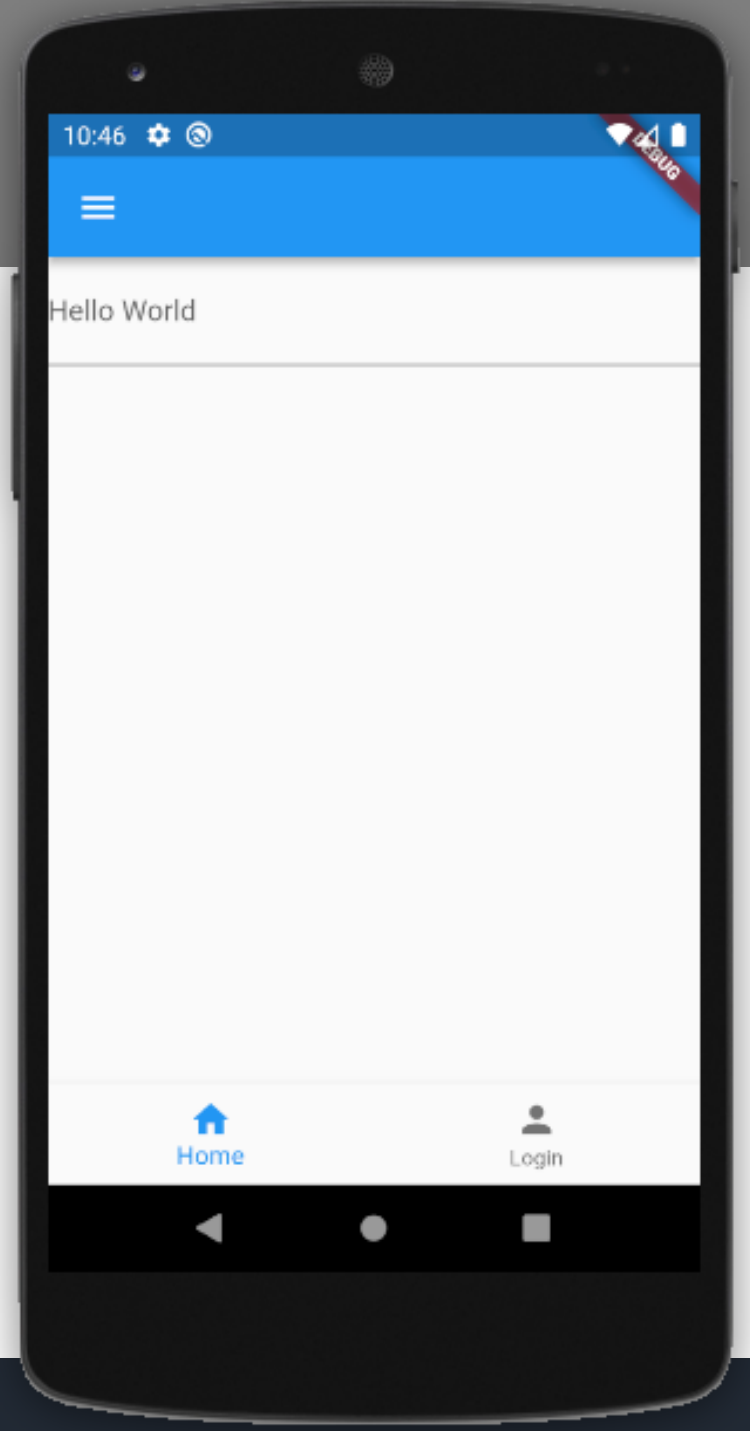
```
home: Scaffold(
  appBar: AppBar(),
  drawer: Drawer(),
  body: const Center(
    child: Text("Hello World")
  ),  // Center
  bottomNavigationBar: BottomNavigationBar(
      items: <BottomNavigationBarItem> [
        BottomNavigationBarItem(
          icon: Icon(Icons.home),
          label: 'Home',
        ),  // BottomNavigationBarItem
        BottomNavigationBarItem(
            icon: Icon(Icons.person),
            label: 'Minha conta',
        ),  // BottomNavigationBarItem
      ],  // <BottomNavigationBarItem>[]
    ),  // BottomNavigationBar
),  // Scaffold
```



Text e TextField

- **Text** e **TextField** são widgets para saída e entrada de texto, respectivamente.
- Ambos possuem opções para customização do texto, mas o **alinhamento** destes widgets depende do widget pai.
- Adicionando um **TextField** (código abaixo) ao **body** do exercício anterior, o resultado será a tela ao lado

```
TextField(decoration: InputDecoration(labelText: 'Hello World')),
```

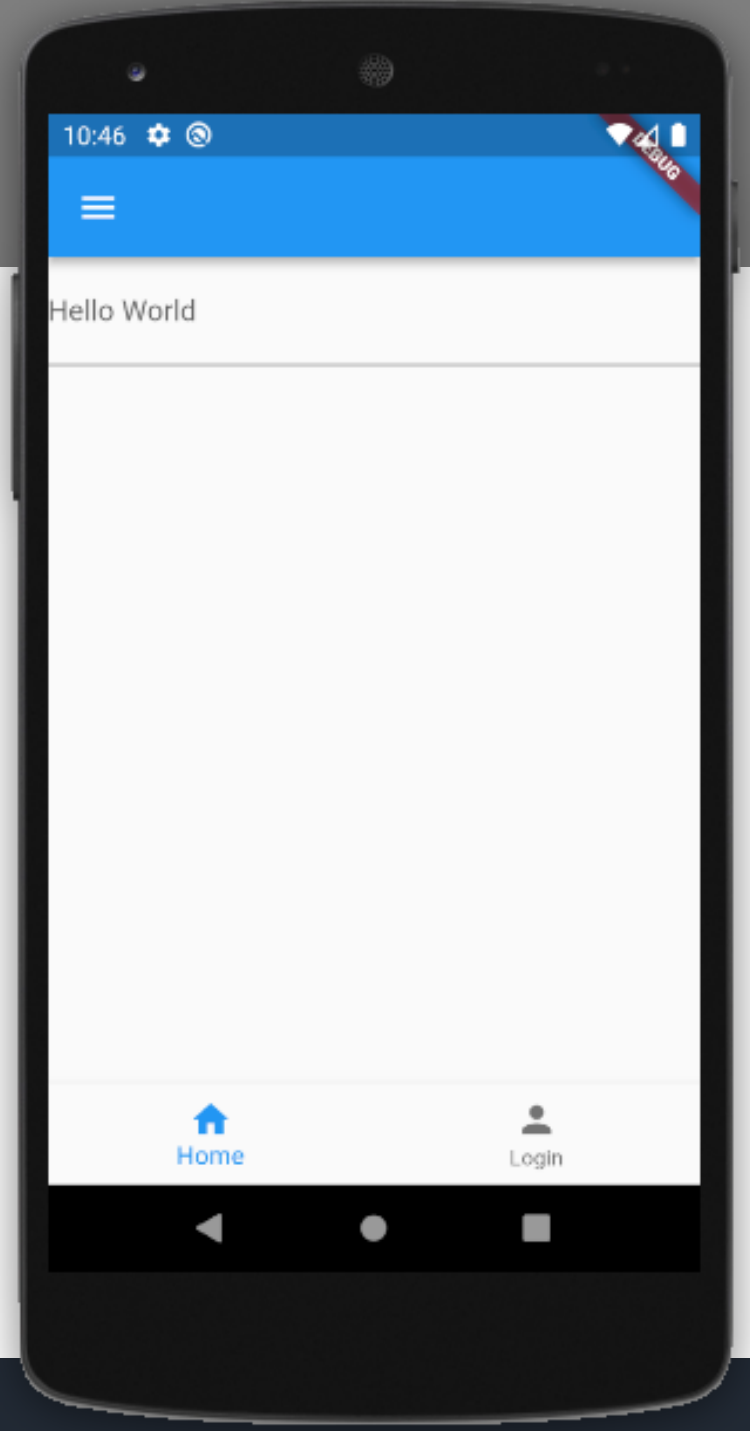


Text e TextField

- **Text** e **TextField** são widgets para saída e entrada de texto, respectivamente.
- Ambos possuem opções para customização do texto, mas o **alinhamento** destes widgets depende do widget pai.
- Adicionando um **TextField** (código abaixo) ao **body** do exercício anterior, o resultado será a tela ao lado

```
TextField(decoration: InputDecoration(labelText: 'Hello World')),
```

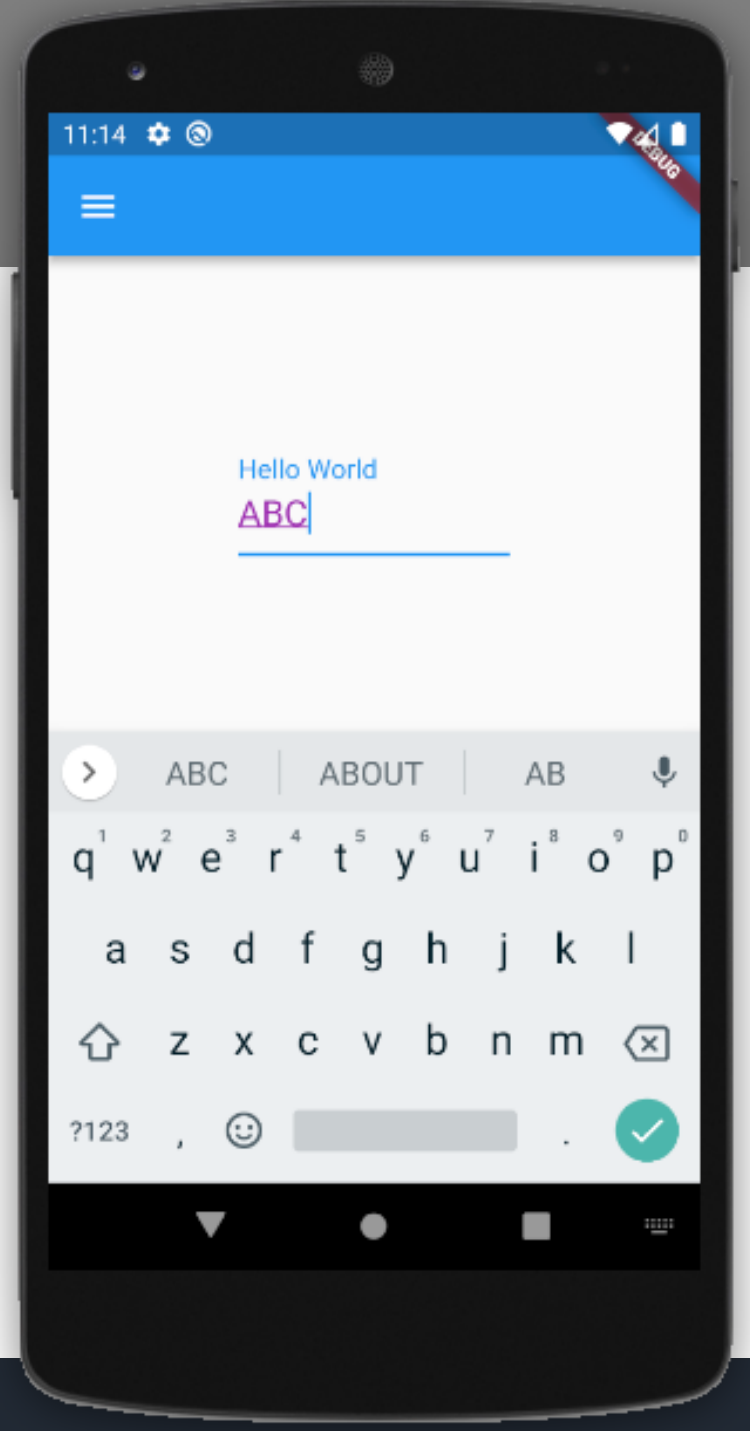
E se quisermos definir um **tamanho** e **alinhamento** para o **TextField**?



Align e Container

- Uma opção é usar um widget **Align** para definir o alinhamento e um widget **Container** para definir um tamanho.

```
body: Align(
    alignment: Alignment.center,
    child: Container(
        width: 150,
        child: TextField(
            decoration: InputDecoration(labelText: 'Hello World'),
            style: TextStyle(color: Colors.purple, fontSize: 20),
        )  // TextField
)),  // Container, Align
```



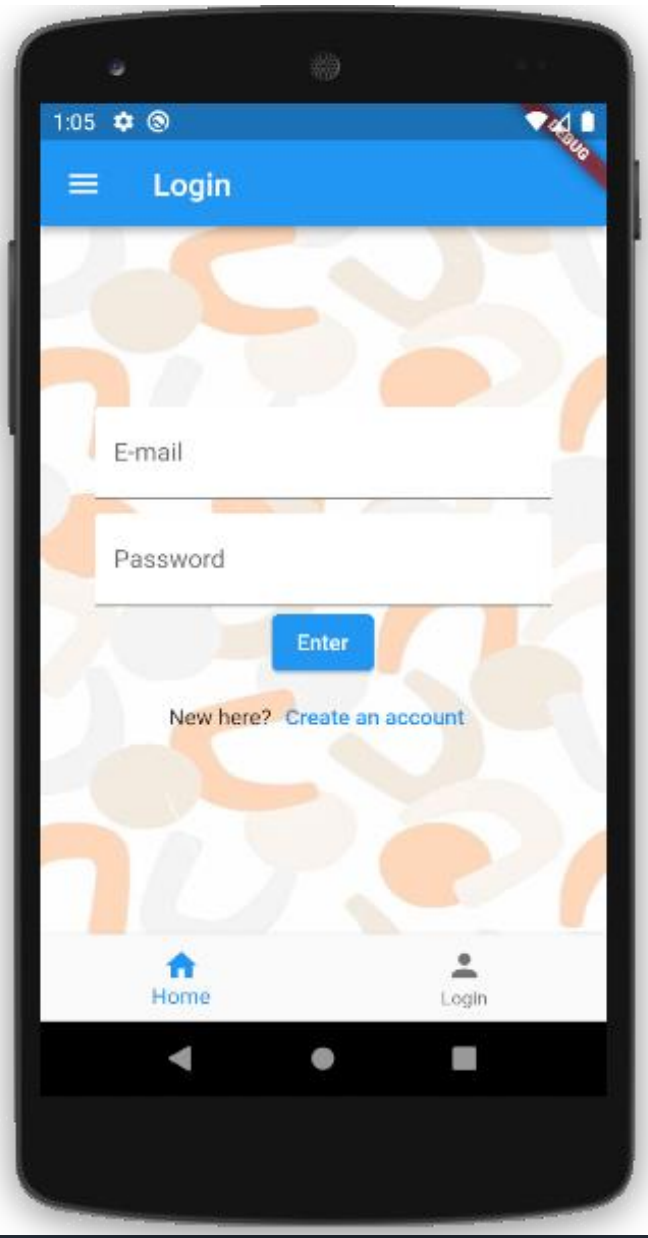
Botões

- Outro elemento comum em interfaces são **botões**. Usaremos o **ElevatedButton** aqui.
- Nele, **onPressed** é um argumento obrigatório e define ações ao pressionar o botão. Por enquanto, definimos uma função anônima vazia.
- Seu **alinhamento** depende do widget pai. Para adicionar texto, utiliza-se como widget filho um **Text**, mas também é possível usar outros.

```
body: ElevatedButton(
    onPressed: (){},
    child: Text('Press here')),
```

Prática – **Entregar**

- Faça uma tela de login similar à tela ao lado, em aparência, espaçamento entre componentes e seus tamanhos.
- Aproveite o exerício para explorar as opções de customização.
- Caso você utilize Row e/ou Column, o alinhamento pode ser feito com MainAxisAlignment.
- “Create an account” é um TextButton com texto azul.



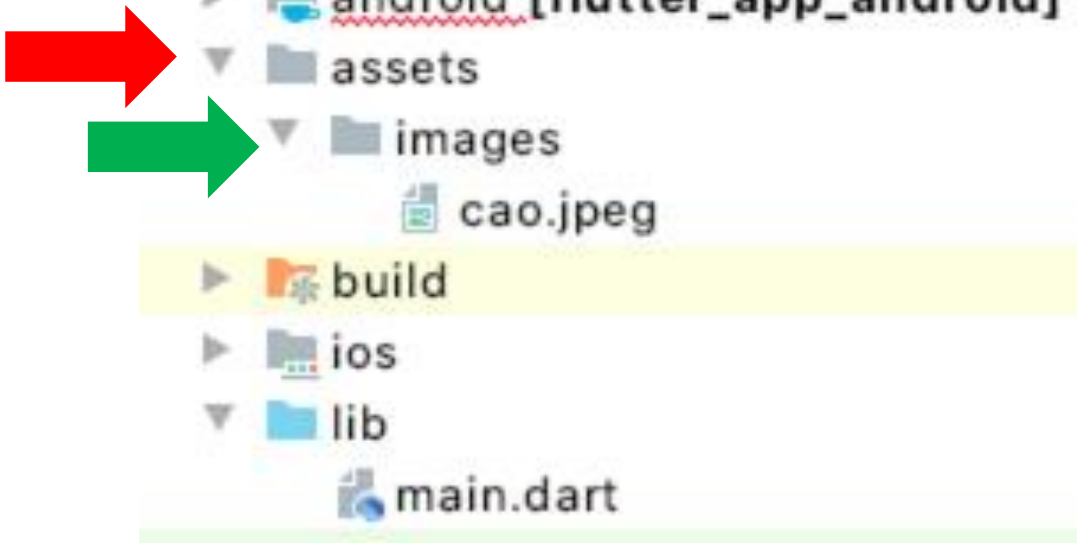
Imagens

Imagens

Agora vamos adicionar uma **imagem**.

No Flutter, existem 4 maneiras de adicionar uma imagem. Nesta aula adicionaremos uma imagem localmente.

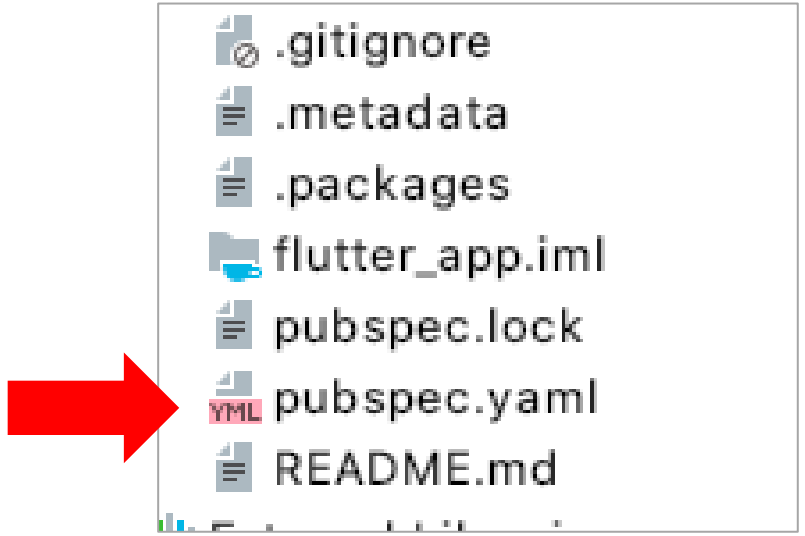
- Crie o diretório assets e o subdiretório imagens, para isso, clique com o botão direito no nome do projeto e vá em **New -> Directory,** em seguida, botão direito em **assets**, **New-> Directory**
- Arraste a(s) imagem(ns) para a pasta *images*



Imagens

- Em seguida, abra o arquivo **pubspec.yaml** e faça as seguintes alterações (como nas imagens ao lado)
- Tire o comentário # de frente de assets e altere o caminho para as imagens, caso tenha mais de uma pode deixar assim: **assets/images/**

Localizando o arquivo:



Arquivo pubspec.yaml

```
# To add assets to your application,
assets:
  - assets/images/
```

Imagens

- Para incluir a imagem no App, inclua o widget **Image** e o método auxiliar **asset** e indique qual é a imagem que será incluída como no exemplo.

Exemplo:

```
_body() {
  return Container(
    color: Colors.white,
    child: Center(
      child: Image.asset('assets/images/cao.jpeg'),
    ), // Center
  ); // Container
}
```

Imagens

- É possível alterar o tamanho da imagem, com **width** e **heigth**
- Podemos usar também o **fit**, com o **BoxFit** cover ou Fill

Exemplo:

```
_img() {
  return Image.asset("assets/images/cao.jpeg",
    width: 350,
    height: 350,
    fit: BoxFit.cover,
  );
}
```

Imagens

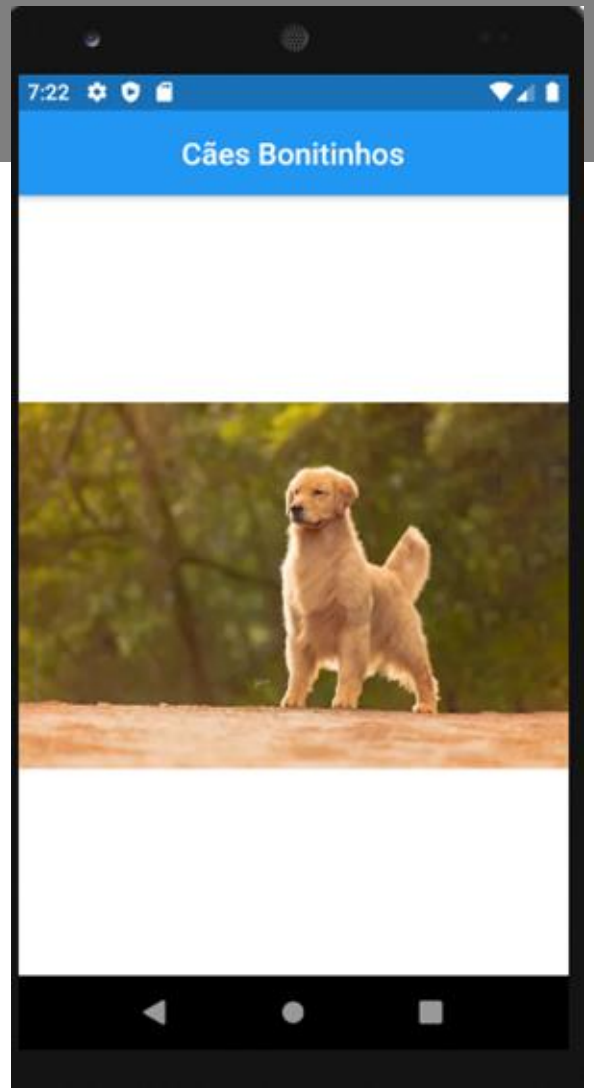
- **BoxFit.cover** corta a imagem, como no Exemplo 1
- **BoxFit.fill** aumenta a imagem distorcendo, como no Exemplo 2

Exemplo 1

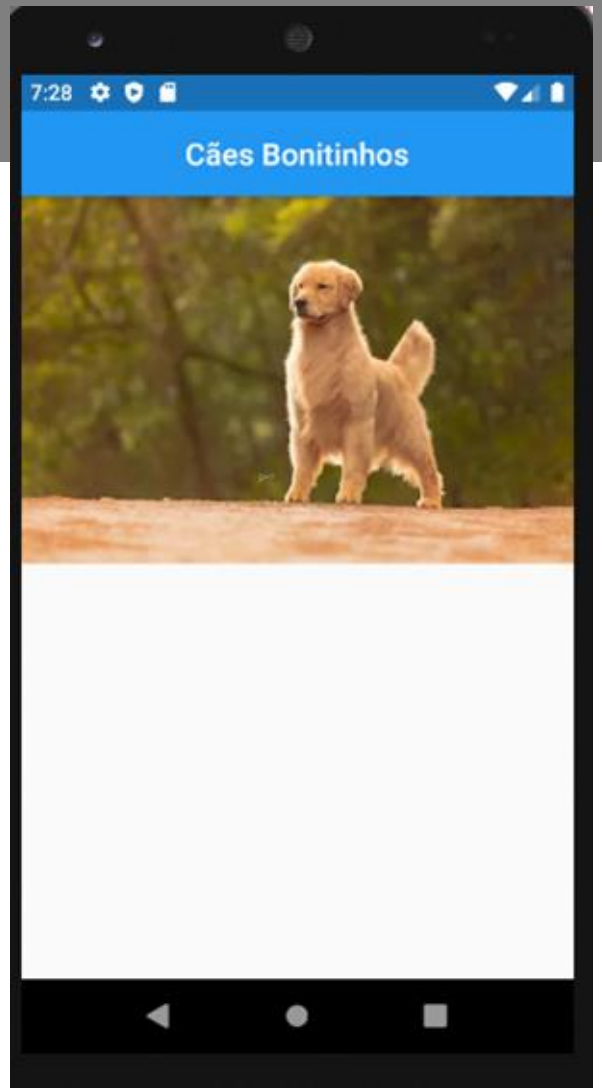
Exemplo 2

Imagens

- Para o exemplo, o widget **Image** foi incluído dentro do widget **Center**, por isso, a imagem do cachorro está no centro (Exemplo 1)
- Se retirar o widget **Center** ela ficará bem no início da tela (Exemplo 2)



Exemplo 1



Exemplo 2

Imagens

- É possível também incluir uma imagem a partir de uma **URL**, para isso, usaremos o método auxiliar **network**, como no exemplo abaixo

Exemplo:

```
_body() {
  return Container(
    color: Colors.white,
    child: Center(
      child: Image.network("https://static.wixstatic.com/media/
      ), // Image.network
    ), // Center
  ); // Container
}
```

SizedBox

- **SizedBox** é um widget de layout para especificar tamanho.
- Assim como **Container**, ele permite somente um widget filho (**child**)
- O widget **SizedBox** permite que seus filhos ocupem o tamanho inteiro da tela se for desejado
- **SizedBox.expand** expande na tela inteira

```
_body() {
  return Container(
    color: Colors.white,
    child: SizedBox.expand(
      child: _img(),
    ), // SizedBox.expand
  ); // Container
}
```

SizedBox em Imagens

```
_body() {
  return Container(
    color: Colors.white,
    child: SizedBox.expand(
      child: _img(),
    ), // SizedBox.expand
  ); // Container
}
_img() {
  return Image.asset("assets/images/cao.jpeg",
    fit: BoxFit.fill,
  );
```

Use tanto o fill, quanto o cover, para ver a diferença